

01 / Mimari kapsam ve temel kararlar

Config-Driven Multi-Robot ROS 2 / Nav2 Architecture

Konfigürasyon güdümlü çok robotlu simülasyon ve web tabanlı filo yönetimi

Teknik mimari dokümanı • Sürüm 1.1 • 8 Ekim 2026

Bu doküman, önceki mimari kararlarını koruyarak Web tabanlı Fleet Configuration yönetimini tasarıma ekler. Hedef; robot sayısını, robot modelini, başlangıç pozunu ve navigasyon profillerini uygulama kodu değiştirmeden yönetebilen, robot türünden bağımsız bir filo mimarisidir.

Korunan tasarım kararları

- Robot tanımlarının kaynağı **fleet.yaml** konfigürasyonudur. Launch sistemi listedeki her robot için gerekli bileşenleri oluşturur; hiçbir katman sabit üç robot varsaymaz.
- Robot kimlikleri serbestçe tanımlanabilir: robot_1, picker_left veya delivery_a. Namespace, topic ve action adları bu kimliklerden türetilir.
- Her robot için model spawn, robot_state_publisher, Nav2 yığını ve battery_simulator instance başlatılır. Aynı executable farklı namespace altında tekrar kullanılır.
- Ortak katmanda map_server, /map ve Fleet Manager bulunur. Robot bazındaki topic, action ve TF adları birbirinden ayrılır.
- Fleet Manager yalnızca robot kimliği, pose, battery ve state bilgileriyle çalışır. Robot türüne özgü ayrıntılar model konfigürasyonu ve uyarlama katmanında tutulur.
- Robot modeli ve robot sayısı gelecekte değiştirilebilir. Model değişimi; model tanımı, spawn uyarlaması ve gerekirse Nav2 parametre profilinin güncellenmesini gerektirir.

MVP sınırı

Web panelinden ekleme, silme ve düzenleme MVP kapsamındadır. Değişiklikler **Apply Configuration** ile kaydedilir ve simülasyon yeni konfigürasyonla yeniden başlatılır. Çalışırken canlı robot ekleme/silme, **stretch goal** olarak bırakılır.

02 / Konfigürasyon modeli ve dosya yapısı

```
fleet_sim/  
  config/  
    fleet.yaml  
    robots/turtlebot3_burger.yaml  
    nav2/nav2_params.yaml  
  launch/  
    simulation.launch.py  
    spawn_fleet.launch.py  
    robot_nav.launch.py  
  worlds/warehouse.sdf
```

Örnek fleet.yaml

```
robots:  
  - id: robot_1  
    type: turtlebot3_burger  
    spawn: {x: 0.0, y: 0.0, yaw: 0.0}  
    nav2_profile: default  
    battery_model: standard  
  - id: picker_left  
    type: turtlebot3_burger  
    spawn: {x: 2.5, y: 4.0, yaw: 1.57}  
    nav2_profile: default  
    battery_model: standard
```

Spawn koordinatları seçilen simülasyon dünya referansında tanımlanır; x ve y metre, yaw radyan cinsindedir. Konfigürasyon, dünya ve harita koordinatlarının uyumunu gözetmelidir.

Alan	Anlamı
id	Benzersiz robot kimliği; namespace ve entity adının kaynağı.
type	Desteklenen robot modeli/uyarlayıcısı anahtarı.
spawn	Başlangıç x, y ve yaw değerleri.
nav2_profile	Nav2 parametre profiline referans.
battery_model	Batarya simülasyonu profiline referans.

Robot sayısı türetilir

```
robot_count = len(config.robots)
```

fleet.yaml içinde ayrı bir robot_count alanı bulunmaz. Launch, kayıt oluşturma, görev atama ve panel sayaçları ilgili robot koleksiyonundan üretilir. Listede 2, 4 veya 7 tanım varsa sistem aynı sayıda robot başlatır.

03 / Robot izolasyonu, TF ve dinamik launch

Robot bazındaki ve ortak bileşenler

```
/<robot_id>/scan          /<robot_id>/planner_server
/<robot_id>/odom           /<robot_id>/controller_server
/<robot_id>/cmd_vel        /<robot_id>/behavior_server
/<robot_id>/battery_state  /<robot_id>/bt_navigator
/<robot_id>/navigate_to_pose
/<robot_id>/lifecycle_manager
```

Ortak: /map, /map_server, /fleet_manager

Namespace, robot id değerinden üretilir. Örneğin picker_left için /picker_left/scan ve /picker_left/navigate_to_pose kullanılır. Robot topic eşlemeleri model konfigürasyonundan alınır; farklı sensör topic adları Fleet Manager sözleşmesini değiştirmez.

TF sözleşmesi

```
map
  robot_1/odom
  robot_1/base_link
  picker_left/odom
  picker_left/base_link
```

Ortak map frame korunur; odom, base ve robotun diğer frame adları robot kimliği ile benzersizleştirilir. Namespace tek başına TF frame_id alanlarını değiştirmez. Model/URDF, odometri, sensör mesajları, lokalizasyon ve Nav2 parametreleri aynı frame adlandırma sözleşmesini kullanmalıdır.

Launch algoritması - kavramsal örnek

```
config = load_and_validate_fleet_config("fleet.yaml")
for robot in config.robots:
    model = resolve_robot_type(robot.type)
    spawn_robot(robot, model)
    launch_robot_state_publisher(robot, model)
    launch_nav2(robot, profile=robot.nav2_profile)
    launch_battery_simulator(robot, robot.battery_model)
    registry.register(robot.id)
```

Bu örnek doğrudan çalıştırılabilir ROS 2 launch kodu değildir; bileşen oluşturma akışını tanımlar. Başlatma, spawn ve gerekli veri/TF hazır olma durumlarına göre sıralanır. Nav2 lifecycle etkinleştirilmesi hazır olma kontrollerinden sonra yapılır.

Fleet Manager kayıtlarını konfigürasyondan dinamik kurar. Görev atama, registry.get_available_robots() gibi bir sorgu üzerinden uygun robotları seçer; robot sayısını veya modelini sabitlemez.

04 / Web tabanlı Fleet Configuration

React panelinde Fleet Configuration ekranı bulunur. Kullanıcı robot ekleyebilir, mevcut robotu düzenleyebilir veya listeden silebilir. Form, id/type/spawn pose/Nav2 profile/battery model alanlarını gösterir; desteklenen tip ve profil seçenekleri backend tarafından sağlanır.

Katmanlar ve sorumluluklar

```
React panel
-> FastAPI
-> Fleet Configuration Service
-> Config Repository
-> fleet.yaml / SQLite
```

Katman	Sorumluluk
React	Robot listesi, form, taslak değişiklikler ve uygulama durumu.
FastAPI	HTTP sözleşmesi, istek ayrıştırma ve hata yanıtları.
Configuration Service	Doğrulama, taslak yönetimi ve Apply işleminin koordinasyonu.
Config Repository	Konfigürasyon okuma/yazma ve tutarlı kalıcılık.

Web, YAML dosyasını doğrudan düzenlemez. Dosya yolu ve saklama ayrıntıları repository arkasında kalır. MVP'de fleet.yaml kalıcı kaynak olarak kullanılır; SQLite gelecekte alternatif repository uygulaması olabilir. İki bağımsız doğruluk kaynağı oluşturulmaz.

Taslak ile kalıcı konfigürasyon ayrımı

CRUD işlemleri sunucu tarafındaki düzenleme taslağını günceller. Formdaki Save, değişikliği taslağa aktarır; çalışan filoyu değiştirmez. **Apply Configuration** bütün taslağı yeniden doğrular, kalıcı repository'ye kaydeder ve yeniden başlatmayı tetikler. Böylece kaydetme ile uygulama sınırı kullanıcı açısından açık kalır.

Panelde görünür durum

Gösterge	Örnek / anlam
Configured robots	5 - düzenlenen robot listesinin uzunluğu.
Running robots	4 - çalışma zamanı registry'sindeki aktif robot sayısı.
Configuration status	Changes pending - taslak henüz uygulanmadı.
Eylem	Apply Configuration - kaydet ve simülasyonu yeniden başlat.

Uygulama tamamlandığında durum Applied olur. Changes pending yalnızca sayaç farkından anlaşılmaz: robot sayısı değişmeden spawn veya profil değişirse de gösterilir. Taslak, kalıcı konfigürasyon ve çalışan sürüm karşılaştırılarak durum belirlenir.

05 / REST API ve backend doğrulamaları

Robot yönetimi endpoint örnekleri

İstek	Davranış / örnek yanıt
GET /api/config/robots	Düzenlenen robot listesini getirir; 200 OK.
POST /api/config/robots	Taslağa robot ekler; 201 Created.
PUT /api/config/robots/{robot_id}	Robot tanımını taslakta günceller; 200 OK.
DELETE /api/config/robots/{robot_id}	Robotu taslaktan siler; 204 No Content.

POST /api/config/robots
Content-Type: application/json

```
{
  "id": "robot_4",
  "type": "turtlebot3_burger",
  "spawn": {"x": 5.0, "y": 1.0, "yaw": 0.0},
  "nav2_profile": "default",
  "battery_model": "standard"
}
```

Düzenleme örneği: PUT /api/config/robots/robot_4 aynı şemadaki tam tanımı gönderir. Silme örneği: DELETE /api/config/robots/robot_4. Kimlik değiştirilebiliyorsa URL mevcut kimliği, gövdedeki id yeni kimliği ifade eder; backend tüm referansları ve benzersizlik kontrolünü birlikte günceller.

Backend doğrulama kuralları

- **Benzersiz ID:** boş olmayan, ROS namespace ve entity adlandırmasına uygun kimlik; liste içinde tekrar kabul edilmez.
- **Desteklenen robot tipi:** type model kataloğunda bulunmalı; Nav2 profile ve battery model referansları çözülmelidir.
- **Spawn geçerliliği:** x/y/yaw sonlu sayılar olmalı; pose dünya sınırları ve başlangıç yerleşimi kısıtlarıyla uyumlu olmalıdır.
- **Çakışma kontrolü:** yalnızca aynı koordinatı karşılaştırmak yeterli değildir. Robot footprint ve güvenlik açıklığı dikkate alınarak diğer robotlar ve bilinen engellerle başlangıç çakışması denetlenir.

Örnek hata sözleşmesi: bulunamayan robot için 404, çakışan kimlik veya konfigürasyon çatışması için 409, geçersiz alanlar için 422. Hata yanıtı alan, kod ve anlaşılır mesaj taşır; panel ilgili form alanını işaretler.

06 / Apply Configuration ve kabul ölçütleri

MVP uygulama akışı

1. Kullanıcı robotları ekler, düzenler veya siler; panel Changes pending gösterir.
2. Apply Configuration bütün taslağı doğrular. Başarısız doğrulamada hata gösterilir ve çalışan simülasyon sürdürülür.
3. Doğrulanmış konfigürasyon repository'ye tutarlı biçimde kaydedilir. Son başarılı konfigürasyon ve sürüm bilgisi yeniden deneme/geri dönüş için korunur.
4. Mevcut görevler durdurulur veya iptal edilir; robot bileşenleri ve simülasyon kontrollü biçimde kapatılır.
5. Kaydedilmiş fleet.yaml tekrar okunur; listedeki N robot, N Nav2 ve N batarya simülatörü başlatılır. Fleet Manager registry yeniden oluşturulur.
6. Hazır olma kontrolleri tamamlanınca çalışan sürüm güncellenir ve Applied gösterilir. Başlatma hatasında Failed durumu ve hata ayrıntısı gösterilir; Applied işaretlenmez.

Apply için önerilen ek sözleşme: POST /api/config/apply. Uzun süren yeniden başlatma işlemi 202 Accepted ile işlem kimliği dönebilir; panel işlem durumunu izler. Bu endpoint, robot CRUD sözleşmesinden ayrı bir uygulama eylemidir. Eşzamanlı Apply işlemleri engellenir; uygulama sırasında yeni düzenlemeler açık bir sürüm/taslak kuralıyla yönetilir.

Stretch goal: runtime hot-add / hot-remove

Hot-add; simülasyona entity ekleme, robot bileşenlerini başlatma, Nav2 lifecycle etkinleştirme ve registry kaydı gerektirir. Hot-remove; görev iptali, lifecycle kapatma, node durdurma, entity silme, TF yayınlarını sonlandırma ve registry temizliği gerektirir. Kısmi hata, kaynak temizliği ve eşzamanlı görev yönetimi nedeniyle bu özellikler MVP sonrasına bırakılır.

Kabul ölçütleri

- Konfigürasyondaki robot sayısı değişince kod değişmeden aynı sayıda instance başlar; özel robot id değerleri desteklenir.
- React üzerinden tüm robot alanları yönetilir; hiçbir tarayıcı işlemi YAML dosyasına doğrudan yazmaz.
- Geçersiz ID, model ve spawn tanımları backend tarafından reddedilir; Apply aşamasında tüm filo yeniden doğrulanır.
- Ekleme/silme ve aynı sayıda robotla alan değişikliği Changes pending üretir. Configured ve Running sayaçları kendi koleksiyonlarından türetilir.
- Apply sonrası yeni filo ve registry doğrulanır; başarısız başlangıç Applied olarak gösterilmez. Robotların topic/action/TF adlarında çakışma oluşmaz.

Kaynak ve kapsam: “Nav2 yığını nedir” başlıklı önceki konuşmadaki mimari kararları ve 8 Ekim 2026 tarihli güncelleme isteği. Mevcut bir PDF eki bulunmadığından bu sürüm erişilebilen konuşma içeriğinden yeniden oluşturulmuştur. API yanıtları, taslak sözleşmesi ve hata akışı bu dokümanda somutlaştırılan tasarım önerileridir; uygulama doğrulaması değildir.